



Mosaic Language Specification

March 25, 2026

Edition 0

Abstract

This document details specifications for the *Mosaic Programming Language*. The official implementation can be found at <https://github.com/jay3332/mosaic>.

Note

This is neither a guide nor documentation. This is a reference used to standardize implementations of the Mosaic programming language ecosystem.

For a proper Mosaic guide, visit ???.

For standard library documentation, visit ???.

For the warning and error index, visit ???.

Warning

This document does not necessarily introduce concepts in order! For example, a topic from Chapter 6 can depend on a concept defined in Chapter 8. What can be guaranteed, though, is that no definitions are circular.

Contributing to this Document

Contributions are welcome! Feel free to propose new language features or fix any inconsistencies with this document.

All edits and contributions should be made to <https://github.com/jay3332/mosaic/tree/main/spec>.

Guidelines

- Make edits to Typst source files; do not modify any PDFs. They are automatically generated.
- A summary of the issue, fix, or feature should be included in the GitHub issue description.
- Edits should be properly formatted using `typstyle`.
- Edits should be properly tested by exporting the document into both PDF and HTML.
 - You should use Typst v0.14 or higher and compile with flag `--features=html`.

Note

A `Makefile` is provided for convenience.

Contents

1 Introduction	3
2 Lexical Structure	4
3 Types	7
4 Variables	9
5 Expressions	10
6 Functions and Closures	10
7 Structs and Enums	13
8 Traits and Generics	13
9 Optionals and Error Handling	13
10 Collections	13
11 Components	13
12 Modifiers	15
13 Templates	16
14 Modules, Packages, and Visibility	18
15 Memory Model	18
16 Concurrency/Async Model	18
17 Annotations and the Preprocessor	18
18 Foreign Function Interface (FFI)	18
Appendix A: Standard Library	19
Appendix B: Native Components	19
Appendix C: Grammar	19

1 Introduction

Overview

Mosaic is a statically typed scripting language designed with the primary purpose of building user interfaces. It is a general-purpose language, however, and can be used for a variety of applications.

Mosaic is designed to be a compiled language which can both compile to native code and transpile to JavaScript ahead of time. For debugging and development purposes, a *subset* of Mosaic can also be compiled and executed JIT¹.

Design Goals

Mosaic is designed with the following goals in mind:

- **UI-first:** Mosaic is designed with user interfaces as a first-class use case. It should be easy to build complex, dynamic user interfaces with Mosaic. Managing state, context, side effects, and asynchronous operations should be easy and intuitive in Mosaic.
- **Native performance:** Mosaic should be able to compile to native code on *most* major platforms. It should also be able to transpile to JavaScript and/or WebAssembly for web applications. It should be able to interoperate with native code and libraries on all platforms.
- **Developer experience:** Mosaic should be easy to debug. It should be easy to publish and distribute Mosaic libraries and applications. Mosaic should have a rich standard library.
- **Safety:** Mosaic should be a “safe” language. It should proactively prevent common programming errors and pitfalls. It should have a strong type system and a robust memory model.

¹Certain implementations may implement an interpreter.

2 Lexical Structure

Encoding

Mosaic source files are encoded in UTF-8. Mosaic source files have the extension `.mos` or `.mosaic`.

Whitespace

Indentation does not have semantic meaning in Mosaic. However, it is *recommended* to use 4 spaces for indentation.

Newlines are used to separate statements. However, they can be omitted if statements are separated by a semicolon (;).

These are all valid Mosaic code snippets:

```
let x = 1
let y = 2
```

```
let x = 1; let y = 2
```

```
{ let x = 1 }
```

```
{ let x = 1
  let y = 2 }
```

Comments

Comments are sections of source code that are ignored by the compiler.

Single Line Comments

Prefixing a line with `//` creates a single line comment:

```
// This is a single line comment
// You can also indent single line comments
```

Block Comments

Wrapping a section of code with `/*` and `*/` creates a block comment:

```
/* This is a block comment
   It can span multiple lines */
```

Block comments nest, so you can have block comments within block comments:

```
/*  
  This is a block comment  
  /* This is a nested block comment */  
  This is still part of the original block comment  
*/
```

Documentation Comments

Documentation comments are a special type of comment used to document code. They are denoted by `///` for *outer documentation comments* and `//!` for *inner documentation comments*.

An outer documentation comment documents the item that follows it:

```
/// This is documents `foo`  
func foo() { ... }
```

An inner documentation comment documents the item that contains it:

```
func foo() {  
  //! This documents `foo`  
}
```

Identifiers

An **identifier** is a name used to refer to an item in Mosaic. An identifier begins with a letter or underscore, followed by any number of letters, digits, or underscores.

Grammar: Identifier

```
letter    ::= "a" | "b" | "c" | ... | "z" | "A" | "B" | "C" | ... | "Z";  
digit     ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9";  
identifier ::= (letter | "_") (letter | digit | "_")*;
```

Extension: Unicode Identifiers

In some implementations, Mosaic identifiers can also contain Unicode letters. In such case, the following would be valid Mosaic code:

```
let café = "coffee";  
let 变量 = "variable";  
let привет = "hello";
```

Grammar: Unicode Identifier

```
unicode_start ::= (* any Unicode character with property "XID_Start"    *);  
unicode_cont ::= (* any Unicode character with property "XID_Continue" *);  
identifier    ::= (unicode_start | "_") unicode_cont*;
```

Keywords

A **keyword** is a word that has special meaning in Mosaic outside of standard identifiers. There are three types of keywords in Mosaic:

- **hard keywords** can only be used in their reserved context.
- **soft keywords** can be used as identifiers if they cause no ambiguity.
- **reserved keywords** are words that cannot be used as identifiers despite not being keywords. They are reserved for potential future use.

Grammar: Keywords

Keywords are represented in the grammar as literal strings, e.g. "`let`".

Literals

A **literal** describes a fixed value in Mosaic source code. Mosaic has the following types of literals:

Integer Literals

Floating Point Literals

String Literals

3 Types

Primitive Types

A **primitive type** is a fundamental type that is built into Mosaic. All primitive types implement the [Copy](#) trait, which means they are by default passed by value.

Integer Types

Type	Size	Range
<code>int</code>	pointer sized	Platform-sized signed integer ²
<code>int8</code>	8 bits (1 byte)	$[-128, 127]$
<code>int16</code>	16 bits (2 bytes)	$[-32768, 32767]$
<code>int32</code>	32 bits (4 bytes)	$[-2^{31}, 2^{31} - 1]$
<code>int64</code>	64 bits (8 bytes)	$[-2^{63}, 2^{63} - 1]$
<code>uint</code>	pointer sized	Platform-sized unsigned integer ²
<code>uint8</code>	8 bits (1 byte)	$[0, 255]$
<code>uint16</code>	16 bits (2 bytes)	$[0, 65535]$
<code>uint32</code>	32 bits (4 bytes)	$[0, 2^{32} - 1]$
<code>uint64</code>	64 bits (8 bytes)	$[0, 2^{64} - 1]$

Note

For transpilation into JavaScript, all integer types that are 32 bits or smaller are represented as JavaScript Numbers (which are technically 64-bit floating point values.) 64-bit integer types are represented as JavaScript BigInts. Mosaic must proactively convert between Number and BigInt during transpilation.

Floating Point Types

All floating point types in Mosaic follow the IEEE 754 standard for binary floating point arithmetic.

Type	Size	Range	Precision
<code>float32</code>	32 bits (4 bytes)	$[1.18 \times 10^{-38}, 3.4 \times 10^{38}]$	Single precision (~6 digits)
<code>float64</code>	64 bits (8 bytes)	$[2.23 \times 10^{-308}, 1.8 \times 10^{308}]$	Double precision (~15 digits)
<code>float</code> ³	64 bits (8 bytes)	Same as <code>float64</code>	Same as <code>float64</code>

²The sizes of `int` and `uint` are target-specific. For native targets, they are 32 bits on 32-bit platforms and 64 bits on 64-bit platforms. For web targets, they are treated as 32-bit integers such that they fit well within the $[-2^{53} + 1, 2^{53} - 1]$ safe integer range for JavaScript numbers.

³`float` is *always* an alias for `float64`.

Boolean Type

The `bool` type has two possible values: `true` and `false`. Its definition is roughly equivalent to:

```
enum bool: uint8 {
    case false = 0
    case true  = 1
}
const true = bool.true
const false = bool.false
```

Character Type

The `char` type represents a single Unicode scalar value. It is a 32-bit type that can represent any *valid* Unicode code point. That is:

- any codepoint within the Unicode range of U+0000 to U+10FFFF, inclusive
- ...**except** for the surrogate code points from U+D800 to U+DFFF, inclusive

Trying to create a `char` which does not contain a valid Unicode scalar value will result in a panic in debug mode, and undefined behavior in release mode.

String Type

A `string` is a length-based UTF-8 encoded string. The contents of a `string` are immutable. Internally, a `string` is represented as a length-based “slice” to a sequence of UTF-8-validated bytes:

```
struct string {
    ptr: *const uint8 // pointer to the first byte of the string
    pub size: uint     // number of bytes in the string
}
```

Note

Like all primitive types, `string` implements the `Copy` trait, so it is passed by value. However, since a `string` is effectively a pointer to the first byte of the string, does not actually copy the string.

Warning

The size field of a `string` represents the number of **bytes** in the string, not the number of characters in the string. To get the number of characters, use the `string.len` method, e.g. `my_str.len()`.

Unit Type

The unit type, denoted by `void`, is a type that has only one value, which is also denoted by `void` (the *void literal*). It is used to indicate the absence of a meaningful value, similar to `()` in Rust.

Compound Types

Array Type

Slice Type

Tuples

Type Aliases

Type Conversions

4 Variables

Local Variables

Global Variables

Constants

Shadowing

References

There are two ways variables can be “passed around” in Mosaic:

- **by value**, which creates a copy of the variable for each use.
- **by reference**, which merely passes around a lookup to the variable for each use.

If a variable is safe to pass by value, it will implement the marker trait `Copy`. Otherwise, it *must* be passed by reference. There are two types of references in Mosaic:

- **strong references**, which contribute to the refcount of the value they point to.
- **weak references**, which do not contribute to the refcount of the value they point to

See Also: [Weak References](#)

To *force* a variable to be passed by reference, we can use the `ref` keyword:

```
let x: int = 10
let y: ref int = ref x // `y` is a strong reference to `x`
```

5 Expressions

Arithmetic and Logical Expressions

Block Expressions

If Expressions

Ternary If Expressions

Loop Expressions

Match Expressions and Pattern Matching

if-let and while-let Expressions

Guard (let-else) Expressions

6 Functions and Closures

Function Declarations

Function Parameters and Arguments

Named Parameters

Default Parameters

Function Overloading

Function Types

Anonymous Functions

An **anonymous function** is a function without a name. They are often used for short, throwaway functions that are not reused elsewhere in the code.

An anonymous function is declared using an expression with the following syntax:

```
func(parameters) { body }
```

If the body consists of a single expression, the braces can be omitted:

```
func(parameters) => expression
```

The types of the parameters can be omitted if they can be inferred from context:

```
let add: func(int, int) -> int
add = func(x, y) => x + y // infer x: int, y: int from the type of `add`
```

Grammar: Anonymous Function

```
anonymous_function ::= "func" "(" parameter_list? ")"
                    ( "{" block "}" | ">=" expression );
parameter_list      ::= parameter ("," parameter)*;
parameter           ::= identifier (":" type)?;
```

Closures and Capturing Variables

A **closure** is a function which captures variables from its surrounding scope. Closures must store references to the captured variables in order to access them, and they must ensure that the captured variables remain valid for as long as the closure itself is valid.

Thus, there are three ways variables can be captured by closures:

- **by strong reference**, which is the default capture mode.
- **by weak reference**, which allows the captured variable to be dropped while the closure is still valid.
- **by value**, which creates a copy of the variable for the closure to use.

The primary way of creating a closure in Mosaic is via an anonymous function. Anonymous functions can capture variables from their surrounding scope, making them closures.

For example, in the following code snippet, the anonymous function captures the variable `x` from its surrounding scope:

```
let x = 10
let add_x = func(y: int) => x + y // `add_x` captures `x`
```

Internally, the above code snippet will desugar into something like this⁴:

⁴Because `x` implements `Copy`, it is actually captured by value in this case.

```

struct add_x_Closure {
    x: ref int // strong reference to `x`
    _f: func(int, int) -> int // the actual function pointer
}

op func add_x.call(y: int) -> int {
    _f(self.x, y)
}

let x = 10
let add_x = add_x_Closure(x: ref x, _f: func(x, y) => x + y)

```

The `add_x` closure holds a valid reference to `x`, increasing its refcount. Thus, we force `x` to live until at least `add_x` is dropped.

Although captures are inferred, they can also be explicitly specified, in which case *only* the specified variables will be captured:

```

let x = 10
let add_x = func[x](y: int) => x + y // `add_x` only captures `x`

```

This allows you to specify the capture mode for each variable:

```

let x = 10
let add_x = func[x](y: int) => x + y // infer capture mode
let add_x = func[owned x](y: int) => x + y // capture by value
let add_x = func[ref x](y: int) => x + y // capture by strong reference (default)

// capture by weak reference
let add_x = func[weak x](y: int) => if let .some(x) = x then x + y else 0

```

Note that if we want to ever mutate a captured variable from within a closure, we must capture it by reference:

```

let mut x = 10
let add_to_x = func[ref mut x](y: int) => x += y // strong mutable reference
// weak mutable reference
let add_to_x = func[weak mut x](y: int) { if let .some(x) = x { x += y } }
let add_to_x = func(y: int) => x += y // infer capture by strong mutable reference

```

Capture Inference

If captures are not explicitly specified, the compiler must infer:

- *which* variables are captured, and
- *how* they are captured (capture mode).

7 Structs and Enums

Struct Declarations

Field Visibility

Enum Declarations

Destructuring

Methods

8 Traits and Generics

Operator Traits

9 Optionals and Error Handling

10 Collections

11 Components

A **component** defines a reusable blueprint for a UI element. Components contain their own state, lifecycle, and rendering logic. They can be composed together to build more complex components and views.

Component Declarations

A component is declared using the `component` keyword:

```
component MyComponent {  
  // component body  
}
```

Properties

Events

State

Side Effects

Computed State

Example

```
component Bold {  
  Text(font: .(weight: .bold)) { children }  
}  
  
component Counter {  
  state count: int = 0  
  computed count_squared: int = count * count  
  
  Row {  
    Text { "$count ^ 2 is " }  
    Bold { "$count_squared" }  
    Button { "+" } on click => count += 1  
  }  
}
```

Context

Lifecycle and Rendering

Component Types

View Blocks

12 Modifiers

A **modifier** is a special type of statement that is only valid within *view* contexts. They

13 Templates

A **template** is a reusable collection of modifiers that can be applied to components. A template is declared using the `template` keyword:

```
template MyTemplate {  
    // template body  
}
```

A template can choose to immediately operate on a component. In such case, we must specify the component type the template is for:

```
template MyTemplate for component(TargetComponent) {  
    // template body  
}
```

This will allow us to use any modifiers specific to `Button` within the template body:

```
component TargetComponent {  
    property fill: color  
    ...  
}  
  
template MyTemplate for component(TargetComponent) {  
    set fill = .blue  
}
```

Finally, we can apply a template to a component using a `use` modifier:

```
component App {  
    TargetComponent(fill: .green)  
    use MyTemplate // apply `MyTemplate` to `TargetComponent`  
  
    // TargetComponent renders with (fill: .blue), since our template  
    // overrides the original (fill: .green)  
}
```

Example

Create and use a template that can be applied to any component with a `fill` property:

```
type HasFill = component(property fill: color)  
template MakeItRed for HasFill {  
    set fill = .red  
}
```

We can also apply a template to the entire component body by applying it at the root of the component declaration, as long as the template is compatible with the current component:

```
template MyTemplate for component(App) { ... }
component App {
  use MyTemplate // apply `MyTemplate` to the entire body of `App`
  ...
}
```

Universal Templates

A template that does not specify operate on a specific component type is called a **universal template**. Universal templates:

- can be applied to any component
- can only use modifiers that are valid for all component types (e.g. `with` and `use`)
- can only inherit from other universal templates
- can be applied at the root of any component declaration

A universal template is declared by omitting the component type:

```
template UniversalTemplate {
  ...
}
```

Template Inheritance

Since applying a template is a modifier just like any other, we can apply a template within another template. This allows us to achieve “template inheritance”:

```
template ChildTemplate for component(TargetComponent) {
  use ParentTemplate // apply `ParentTemplate` to `ChildTemplate`
  // additional modifiers
}
template ParentTemplate {}
```

Note that:

- There cannot be any circular dependencies between templates. A template cannot apply itself, either directly or indirectly.
- We can only inherit from templates that operate on the same or a wider range of component types. *In the above example, `ParentTemplate` is a root template that can be applied to any component, so it is valid for `ChildTemplate` to apply it.*

Parameterized Templates

A **parameterized template** is a template that takes parameters, similar to a function:

```
template ParameterizedTemplate(c: color) for component(TargetComponent) {  
  set fill = c  
}
```

They may also take stateful parameters:

```
template ParameterizedTemplate(count: ref int) for component(TargetComponent) {  
  set fill = if count > 5 then .red else .green  
}
```

14 Modules, Packages, and Visibility

Visibility Modifiers

15 Memory Model

Weak References

A **weak reference** is a reference that does not contribute to the refcount of the value it points to. This allows for the possibility of the value being dropped while there are still weak references to it. That is, a weak reference can become invalid at runtime, and we must check for its validity before using it.

To create a weak reference to a value, we can use the `weak` keyword:

```
let x = 10  
let y = weak x // `y` is a weak reference to `x`
```

Weak references have the type `weak T`, where `T` is the type of the value being referenced.

16 Concurrency/Async Model

17 Annotations and the Preprocessor

Conditional Compilation

18 Foreign Function Interface (FFI)

Mosaic supports interoperability with native code and libraries via a **foreign function interface**.

Supported Calling Conventions

Convention	Description
<code>extern("c")</code>	Call the function with the C calling convention (CDECL).
<code>extern("js")</code>	The function corresponds to a JavaScript function. (web target only)
<code>extern("objc")</code>	The function corresponds to a Swift/Objective-C method. (Apple platforms)
<code>extern("jni")</code>	The function corresponds to a Java method. (Android target only)

Appendix A: Standard Library

Appendix B: Native Components

```
native component Button {  
    property fill: Fill  
    property stroke: Stroke  
  
    event click(e: ClickEvent)  
}
```

Appendix C: Grammar